Stoquify Migration Control-Plane Implementation Report

Date: 2026-08-28 Workspace: `E:\ohada saas\Focused projects\stoquify` Outcome: **local control plane remediated and verified; production promotion remains correctly blocked on external evidence.**

Executive outcome

The migration blocker was not one corrupt local database. It was a control-plane design problem: immutable history, target-specific execution risk, baseline adoption, and release evidence were being mixed into one global decision.

The repository now has a deterministic 80-migration catalog, a versioned hash contract, target-pending destructive-risk evaluation, fail-closed baseline path selection, separated historical-approval/release-snapshot semantics, a safe fresh-replay runner, and updated operating guidance. A real schema mismatch discovered during replay was fixed forward with `20260828190000_align_onboarding_risk_reasons_default`; no applied migration was edited.

The configured local database is 80/80 and Prisma reports it up to date. A newly created replay-only database applied all 80 migrations, a second deploy was a no-op, every checksum matched, and unexpected structural drift was zero. Six focused suites passed all 49 tests.

This does **not** authorize production. No remote target inventory, backup/PITR proof, restore rehearsal, transaction failure injection, functional auth/accounting smoke, human approval, or independent checker decision was supplied or invented.

What was actually wrong

Issue	Evidence	Permanent treatment
Historical destructive SQL blocked current targets	The baseline bridge has 13 destructive findings although the configured local target had already applied it with a matching checksum	Full history is checked globally; destructive authorization is evaluated only for the exact target-pending set
Applied migration immutability was implicit	No deterministic accepted-hash manifest covered every migration	Versioned catalog records full name, byte length, raw hash, canonical LF hash, and canonical CRLF hash; mutation and missing/unmanifested entries fail closed
Duplicate timestamp prefix could break timestamp-only tools	Two exact migrations use `20260818120000`	Those two full names are grandfathered; every new collision fails
Existing and empty baseline paths were conflated	The baseline SQL declares itself baseline-only	Existing targets with prior history select manual resolve-only adoption and can never auto-execute; genuinely empty targets remain approval/evidence gated
Historical approval was coupled to mutable source	Packet bindings mixed immutable SQL with package/schema/application snapshots	SQL/operation identity is immutable; source state is a separate release snapshot

Issue	Evidence	Permanent treatment
Hash identities were ambiguous	Newer review used normalized LF while an older packet used raw bytes	`stoquify-migration-sql-sha256-v1` makes canonical LF the approval identity and raw SHA-256 transport identity
Replay tooling could not use one safe database namespace	Bootstrap and certification accepted different prefixes	Both accept an explicit local `stoquify_replay_*` target; the runner creates once, deploys twice, and certifies without retaining the URL
Current migration chain had one real schema mismatch	First replay reported `riskReasons DROP DEFAULT` as unexpected drift	A narrow forward-only migration drops the database default to match `schema.prisma`; the full 80-chain replay then passed

Implemented controls

- `scripts/prisma-migration-catalog-gate.js` and `prisma/migration-catalog-manifest.json` enforce immutable catalog identity.
- `prisma/migration-catalog-exceptions.json` limits duplicate-timestamp grandfathering to two exact names.
- `scripts/prisma-migration-history-health-check.js` now distinguishes deployment-ready pending migrations from history defects.
- `scripts/prisma-production-migration-gate.js` queries target history before production authorization, scopes risk to pending migrations, and blocks automatic baseline execution on existing databases.
- `scripts/prisma-destructive-migration-evidence-gate.js` separates immutable approval bindings from mutable release-snapshot bindings.
- `scripts/prisma-fresh-replay-runner.js` safely creates or read-only certifies explicit local replay databases.
- `scripts/prisma-fresh-replay-certification.js` accepts the replay namespace and fails on unexpected structural drift.
- `docs/operations/runbooks/prisma-production-migrations.md` documents the release boundaries and controlled resolve-only path.
- `package.json` exposes catalog and fresh-replay commands and includes the catalog gate in policy checks.

Verification result

Passed

- Prisma schema validation.
- Deterministic catalog: 80 non-empty migrations, no mutation, no unmanifested or missing entry.
- Configured local history: 80 applied, zero missing, unfinished, rolled back, unknown, duplicate-success, or checksum mismatch.
- Prisma local migration status: database schema up to date.
- Fresh empty replay: 80 applied, second deploy no-op, zero unexpected structural drift.
- Local safety readiness: 10/10 checks, zero active-scope destructive findings, zero blockers.
- Focused automated tests: 49/49 passed across six suites.
- Secret safety: database URLs and credentials were neither printed into nor retained by evidence artifacts.

Failed and remediated

- Initial 79-migration replay failed schema certification on one unexpected default. The mismatch was corrected by a forward-only 80th migration, and a new full replay passed.
- The first replay proof write was interrupted by filesystem permission enforcement after deployment succeeded. A tested `certify-existing` mode completed diagnostic capture without recreating or deleting that database.

Skipped or not applicable

- Existing-database resolve rehearsal: not applicable to the configured local database because the baseline is already recorded with matching history; still mandatory for any remote target where the baseline is pending.
- Frontend, accessibility, localization, commercial packaging, billing, and customer-facing workflow changes: not applicable; no user-facing or entitlement contract changed.
- Dependency upgrade: not performed. The Prisma adapter version difference remains hygiene, not a proven cause.

Externally blocked

- Complete remote environment census and named database owners.
- Backup/PITR attestation and isolated restore rehearsal.
- Transaction failure injection at the destructive boundary.
- Functional Better Auth/session and accounting/ledger smoke evidence on the release candidate.
- Maker attestation, independent checker decision, and any exact-hash destructive approval.
- Clean frozen release candidate and immutable production release manifest.

Multidisciplinary review disposition

Reviewer lens	Disposition
Enterprise/platform architecture	Material: control boundaries and deployment ordering were corrected
Backend/integration	Material: target-derived pending set and fail-closed execution contract
Data/database/migration	Material: catalog immutability, checksum mapping, replay, fix-forward drift correction
Security/IAM/privacy	Material but production-unverified: baseline affects auth/accounting; functional smoke remains external
Finance/OHADA/internal controls	Material but not certified: destructive accounting baseline still requires target evidence and checker approval
Audit/evidence/data quality	Material: immutable versus release-scoped evidence semantics are separated
QA/release assurance	Material: 49 focused tests and current replay proof; external failure/restore/domain smoke still open
SRE/DevSecOps	Material: deployment preflight and safe local/preview defaults; remote backup/observability ownership unresolved
Frontend/design/accessibility/localization	Not applicable: no operational UI contract changed

Reviewer lens	Disposition
Packaging/billing/growth/customer success	Not applicable: no module entitlement or commercial contract changed

Best permanent operating route

1. Merge only after reviewing the catalog manifest, forward migration, focused tests, and this evidence set on a clean candidate.
2. Make catalog integrity and local safety mandatory CI gates for every migration change.
3. For each real target, collect a redacted census and run read-only full-history health before selecting a path.
4. If no migrations are pending, do nothing; historical destructive SQL must not block the target.
5. If the baseline bridge is pending on any target with completed history, stop automation and use the independently approved resolve-only rehearsal path.
6. If the target is genuinely empty, execute only after exact destructive approval plus zero-row, backup/restore, failure-injection, and auth/accounting evidence.
7. Deploy only the verified pending set, then require history status, schema-drift classification, functional domain smokes, and evidence archive before promotion.
8. Use additive expand/backfill/verify/contract migrations whenever a target matches neither empty execution nor compatible resolve-only adoption.

Evidence index

- `what-next/migrations/MIGRATION_CONTROL_PLANE_DECISION_RECORD_2026-08-28.md`
- `what-next/migrations/migration-environment-census-2026-08-28.json`
- `what-next/migrations/migration-catalog-health-2026-08-28.json`
- `what-next/migrations/local-migration-history-health-2026-08-28.{md,json}`
- `what-next/migrations/migration-control-plane-readiness-2026-08-28.{md,json}`
- `what-next/migrations/current-fresh-replay-certification-2026-08-28.{md,json}`
- `what-next/migrations/destructive-migration-evidence-separation-2026-08-28.{md,json}`
- `what-next/migrations/migration-control-plane-local-evidence-manifest-2026-08-28.json`

Final decision: **the local migration control plane is ready for review; production execution remains unauthorized until the named external evidence is supplied and independently approved.**